

Увод в програмирането - основни конструкции

Ивайло Андреев + gemini 3 Flash, chat GPT 5.4

25 май 2026

Съдържание

1	Принципи на структурното програмиране	2
2	Управление на изчислителния процес	2
2.1	Условни оператори	2
2.2	Оператори за цикъл	3
3	Променливи	4
3.1	Локални и глобални променливи	4
3.2	Инициализация и оператор за присвояване	4
4	Функции и процедури	5
4.1	Предаване на параметри	5
4.2	Анонимни функции (Ламбда функции)	6
4.3	Типове и проверка за съответствие на тип	6

1 Принципи на структурното програмиране

Процедурната парадигма служи като основа за разбирането на съвременните подходи и е неизменна част от изучаването на алгоритми и структури от данни. Програмирането може да бъде разделено на две основни парадигми:

- **Императивно програмиране:** Описва *как* се изпълнява дадена задача, чрез поредица от инструкции, които променят състоянието на програмата (например чрез променливи и команди за контрол на потока). То включва процедурното и обектно-ориентираното програмиране.
- **Декларативно програмиране:** Описва *какво* трябва да се постигне, без изрично да се указват стъпките за изпълнение (например функционално програмиране и логическо програмиране).

Преди структурното програмиране програмите често са били изграждани като линейна последователност от инструкции с множество безусловни преходи (`goto`) за реализиране на условна логика и повторение на операции. Това е водело до трудно четим и труден за поддръжка код („spaghetti code“).

Структурното програмиране въвежда организиране на програмата чрез ясно дефинирани контролни конструкции и блокове:

- последователност;
- условни конструкции (`if`, `switch`);
- цикли (`for`, `while`, `do-while`);
- функции и процедури.

Основен принцип е избягването на хаотични преходи чрез `goto` и изграждането на програми с ясен и предвидим контролен поток. Това подобрява четимостта, поддръжката и надеждността на кода. Блоковете могат да имат собствен `scope` и локални променливи, което улеснява организирането на програмата.

2 Управление на изчислителния процес

Управлението на изчислителния процес се осъществява чрез инструкции, които определят реда на изпълнение на действията. Програмата се състои от изрази (които пресмятат и връщат стойност) и оператори (които извършват действия). Блок от код се огражда с къдрави скоби `{}`.

2.1 Условни оператори

Условните оператори позволяват разклоняване на логиката. Използват се `if`, `if-else` и `switch` за многозначно разклоняване.

```

1 // Example of conditional statements and basic syntax
2 int x = 10;
3 if (x > 5) {
4     // Executed if the condition is true
5     x--;
6 } else {
7     // Executed if the condition is false
8     x++;
9 }
10
11 // Inline ternary conditional choice
12 int y = (x == 9) ? 1 : 0;
13 // Equivalent using if-else:
14 // int y;
15 // if (x == 9) { y = 1; } else { y = 0; }
16
17 // Switch selector statement - without break, execution falls through to
    the next cases
18 switch (y) {
19     case 0:
20         // Handle case 0 block
21         break; // Stop fall-through
22     case 1:
23         // Handle case 1 block
24         break;
25     default:
26         // Optional catch-all block
27         break;
28 }

```

2.2 Оператори за цикъл

Циклите осигуряват многократно изпълнение на код. Те се делят на цикли с предусловие (**while**), цикли с постусловие (**do-while**) и цикли с брояч (**for**).

```

1 // 1. Pre-condition loop (evaluates condition before running body)
2 int i = 0;
3 while (i < 5) {
4     // Loop code statements
5     i++;
6 }
7
8 // 2. Post-condition loop (guaranteed to execute at least once)
9 int j = 0;
10 do {
11     // Executes body, then evaluates condition
12     j++;
13 } while (j < 5);
14
15 // 3. Counter-controlled loop iteration (k is local to the for loop)
16 for (int k = 0; k < 5; k++) {
17     // Initialization, condition check, and step execution
18 }

```

```

19
20 // break terminates the loop; continue skips the rest of the current
    iteration
21 for (int i = 0; i < 10; i++) {
22     if (i == 3) continue; // Skip i == 3
23     if (i == 7) break;    // Stop at i == 7
24 }

```

3 Променливи

Променливата е наименувана област от паметта, която съхранява стойност от определен тип. Всяка променлива има адрес, съответстващ на позицията ѝ в паметта.

3.1 Локални и глобални променливи

Основните разлики между променливите се свеждат до техния обхват (видимост) и време на живот:

- **Локални променливи:** Дефинират се вътре във функция или блок `{}`. Те се създават при достигане на дефиницията им и се унищожават при излизане от блока (разполагат се в **Стека**). Видими са само в този блок.
- **Глобални променливи:** Дефинират се извън всички функции, в началото на файла. Те съществуват през цялото време на изпълнение на програмата (разположени в **Data/BSS сегмента**) и са достъпни от всяка функция в рамките на файла.

3.2 Инициализация и оператор за присвояване

- **Оператор за присвояване (=):** Променя текущата стойност на вече съществуваща променлива, като оценява израза отдясно и го записва в паметта на променливата отляво.
- **Инициализация:** Задаване на начална стойност на променливата в момента на нейното деклариране. Важно е да се отбележи, че неинициализираните локални променливи съдържат произволни данни от паметта (т.нар. "бок-лук"), което води до грешки и недефинирано поведение, докато глобалните се инициализират автоматично с 0.

```

1 int globalVar;           // Global variable, auto-initialized to 0
2
3 int main() {
4     int a;               // Local variable, uninitialized (contains garbage)
5     a = 5;               // Assignment: changes the current value
6     int b = 10;          // Initialization: sets value at declaration
7 }

```

4 Функции и процедури

Функцията е самостоятелен блок от код, който решава подзадача и може да бъде извикан многократно. В C++ процедурите се представят като функции с тип на връщане `void`.

- **Формални параметри:** Променливите, изброени в заглавието на функцията.
- **Фактически параметри (Аргументи):** Реалните стойности или променливи, които се подават в скобите при извикването.

Предимства на функциите:

- Повторно използване на код;
- Подобрява четимостта и структурата;
- Улеснява поддръжката и тестването;
- Подпомага модулността и разделянето на задачите на логически части.

4.1 Предаване на параметри

Основните механизми за предаване на параметри:

- **Предаване по стойност (Pass by Value):** Създава се локално копие на аргумента в стековата рамка на функцията. Промените вътре във функцията не влияят на оригиналната променлива.
- **Предаване по референция (Pass by Reference):** Подава се референция към оригиналната променлива. Промените вътре във функцията влияят директно на оригинала.
- **Предаване по указател (Pass by Pointer):** Подава се адресът на оригиналната променлива. Промените чрез дерефериране на указателя влияят на оригинала.

```
1 // Demonstrating parameter passing models
2 void modifyValue(int num) {
3     // Pass by value: changes only the local stack copy
4     num = 100;
5 }
6
7 void modifyByReference(int& num) {
8     // Pass by reference: changes the original argument
9     num = 100;
10 }
11
12 void modifyByPointer(int* num) {
```

```

13 // Pass by pointer: dereference to change the original
14 *num = 100;
15 }
16
17 int main() {
18     int x = 5;
19
20     modifyValue(x);           // x remains 5
21     modifyByReference(x);     // x becomes 100
22     modifyByPointer(&x);      // x becomes 100
23
24     return 0;
25 }

```

Референцията е синтактична захар за указател. Основните разлики между тях:

- Референцията е винаги валидна — тя трябва да бъде инициализирана при декларацията; указател може да е `nullptr`.
- Референцията не може да бъде пренасочвана — след инициализация тя винаги сочи към същия обект; указател може да се пренасочи към друг обект.
- Референцията не може да е нулева — `nullptr` референция е недефинирано поведение.

4.2 Анонимни функции (Ламбда функции)

Функциите могат да бъдат и анонимни — без име, или наричани още ламбда функции. Обща дефиниция:

[<прихващащ клас>] (<формални параметри>) <тяло>

Прихващащият клас може да е [&] или [=]:

- [&] — променливите са прихванати по референция;
- [=] — променливите са прихванати по стойност (по подразбиране).

Тялото се очаква да завършва с израз с `return`.

Пример: сортиране на масив с анонимна функция:

```

1 int arr[] = {3, 1, 2};
2 std::sort(begin(arr), end(arr), [&](int a, int b) { return a < b; });

```

4.3 Типове и проверка за съответствие на тип

C++ е статично типизиран език. Компиляторът извършва строга проверка за съответствие на типовете (**Type Checking**) по време на компилация. Когато подадем аргумент на функция, компилаторът проверява дали типът му съвпада с дефинирания формален параметър. При несъвпадение се прави опит за автоматично (неявно) преобразуване на типа, а ако такова е невъзможно, компилацията се прекъсва с грешка за несъвместимост.

Основните примитивни типове в C++ включват:

- `int` — цяло число (обикновено 4 байта);
- `double, float` — числа с плаваща запетая;
- `char` — единичен символ;
- `bool` — булева стойност (`true` / `false`);
- `void` — липса на тип (използва се за функции, които не връщат стойност).

Понеже C++ е статично типизиран, той поддържа **предефиниране на функции по параметри (function overloading)** — няколко функции могат да имат едно и също име, но различни типове или брой параметри. Компиляторът избира подходящата версия въз основа на типовете на аргументите.

```
1 int    add(int a,    int b)    { return a + b; }  
2 double add(double a, double b) { return a + b; }
```